

D213: Advanced Data Analytics

Task 1: Time Series Modeling

Hannah R. Bergman

WGU Email: hbergm3@wgu.edu

Part I: Research Question.....	2
A1. Research Question.....	2
A2. Goals of the Data Analysis.....	2
Part II: Method Justification.....	3
B. Key Assumptions of Time Series Modeling.....	3
Stationarity.....	3
Autocorrelation.....	3
Linearity.....	3
No Outliers.....	3
Part III: Data Preparation.....	4
C1. Visualizing the Realization of the Time Series.....	4
C2. Time Step Formatting.....	5
Time Step Formatting.....	5
Gaps in Measurement.....	5
Length of the Sequence.....	5
C3. Stationarity Evaluation.....	5
C4. Data Preparation Steps.....	6
Step 1. Convert 'Day' Column to 'Date' (DateTime object).....	6
Step 2. Check for Missing Values.....	7
Step 3. Check for Outliers & Other Anomalies.....	7
Step 4. Split the Dataset into Training & Test Sets.....	8
C5. Cleaned Dataset.....	9
Part IV: Model Identification & Analysis.....	9
D1. Annotated Findings & Visualizations.....	9
1. Seasonality.....	9
2. Trends.....	10
3. Autocorrelation Function (ACF).....	10
4. Spectral Density.....	11
5. Decomposed Time Series.....	11
6. Confirmation of Lack of Trends in Residuals of Decomposed Series.....	12
D2. ARIMA Model.....	13
D3. Forecast.....	14

D4. Output & Calculations of the Analysis.....	14
D5. Code.....	22
Part V: Data Summary & Implications.....	22
E1. Data Analysis Results.....	22
Selection of an ARIMA Model.....	22
Prediction Interval of the Forecast.....	22
Justification of the Forecast Length.....	22
Model Evaluation Procedure & Error Metric.....	23
E2. Visualization of the Forecast of the Final Model.....	23
E3. Recommended Course of Action.....	24
Part VI: Reporting.....	24
F. Jupyter Notebook Report.....	24
G. Data & Code Sources.....	24
H. References.....	25

Part I: Research Question

A1. Research Question

What is the expected pattern of hospital revenue over the next 120 days, and how confident can we be in those predictions?

A2. Goals of the Data Analysis

This data analysis aims to model historical hospital revenue trends by identifying underlying patterns, long-term movements, and seasonal variations that could impact future financial performance. The primary goal is to construct and validate a time series forecasting model capable of accurately capturing both the trend and seasonal components present in daily revenue data. The model will be utilized to generate a 120-day forecast of hospital revenue, offering practical insights for short-term strategic planning.

To ensure the reliability of the projections, the analysis will quantify the forecast accuracy by comparing the model's predictions against actual revenue data held in a test set. Performance metrics, such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), will be used to evaluate the model's predictive capability. We'll also assess the confidence in the generated forecast by calculating and presenting 95% confidence intervals, providing a statistical range within which future revenue values are likely to occur.

Part II: Method Justification

B. Key Assumptions of Time Series Modeling

Stationarity

The statistical properties of a stationary time series, including mean, variance, and autocorrelation, are assumed to remain constant over time. This means that the data doesn't display trends or seasonality, and that the autocorrelation function (ACF) is stable over time (Statistics Solutions, 2021).

Autocorrelation

Time series modelling assumes the time series exhibits meaningful autocorrelation, meaning past values help predict future values (Kuo, 2024).

Linearity

Many time series models, like ARIMA, assume a linear relationship between the current value and its past values (Wainaina, 2023).

No Outliers

The analysis and model results are based on the assumption that there are few or no outliers, as these can significantly skew the results.

Part III: Data Preparation

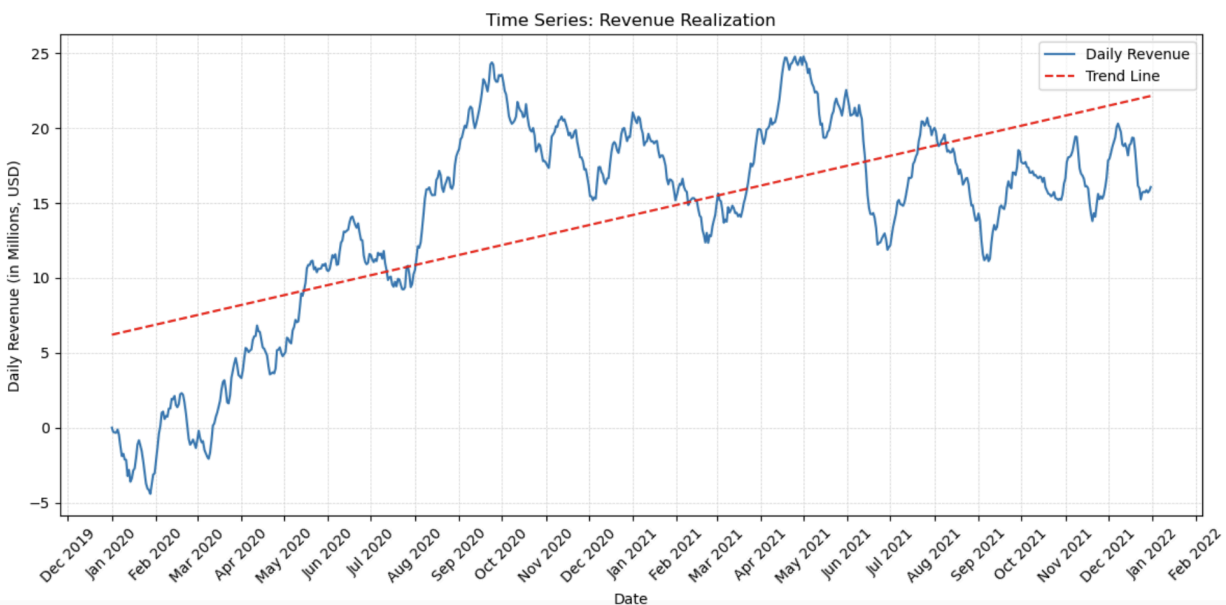
C1. Visualizing the Realization of the Time Series

```
# Plotting the time series line graph
plt.figure(figsize=(12, 6))
plt.title("Time Series: Revenue Realization")
plt.xlabel("Date")
plt.ylabel("Daily Revenue (in Millions, USD)")
plt.grid(True, color='lightgray', linestyle='--', linewidth=0.5)

# Plot the revenue line
plt.plot(df.index, df['revenue'], label='Daily Revenue')

# Add trend line
x = mdates.date2num(df.index.to_pydatetime())
y = df['revenue'].values
z = np.polyfit(x, y, 1)
p = np.poly1d(z)
plt.plot(df.index, p(x), "r--", label='Trend Line')

# Make x-axis look nicer
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
plt.gca().xaxis.set_major_locator(mdates.MonthLocator(interval=1))
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



After cleaning and appropriately labeling our initial dataset, we're able to visualize the realization of the time series using a line graph, which shows our hospital's daily revenue in millions of dollars (USD) over the course of two years (Jan 1st, 2020 - Dec 31st, 2021). Our trend line is upward sloping, indicating that on average, revenue has increased over this two-year period. Despite short-term fluctuations, the long-term outlook shows positive growth in revenue. That being said, our revenue line also clearly shows repeated rises and dips, suggesting possible seasonal or cyclical behavior. Peaks seem to appear consistently around mid-year (June-August) in both 2020 and 2021, and dips are noticeable around February-March and October-November, which could indicate slower business periods or off-season effects.

C2. Time Step Formatting

Time Step Formatting

The data is a time series of daily revenue in millions of USD, with each data point representing one day's revenue. The datetime index **increases by one day for each observation**, from January 1, 2020 to December 31, 2021.

Gaps in Measurement

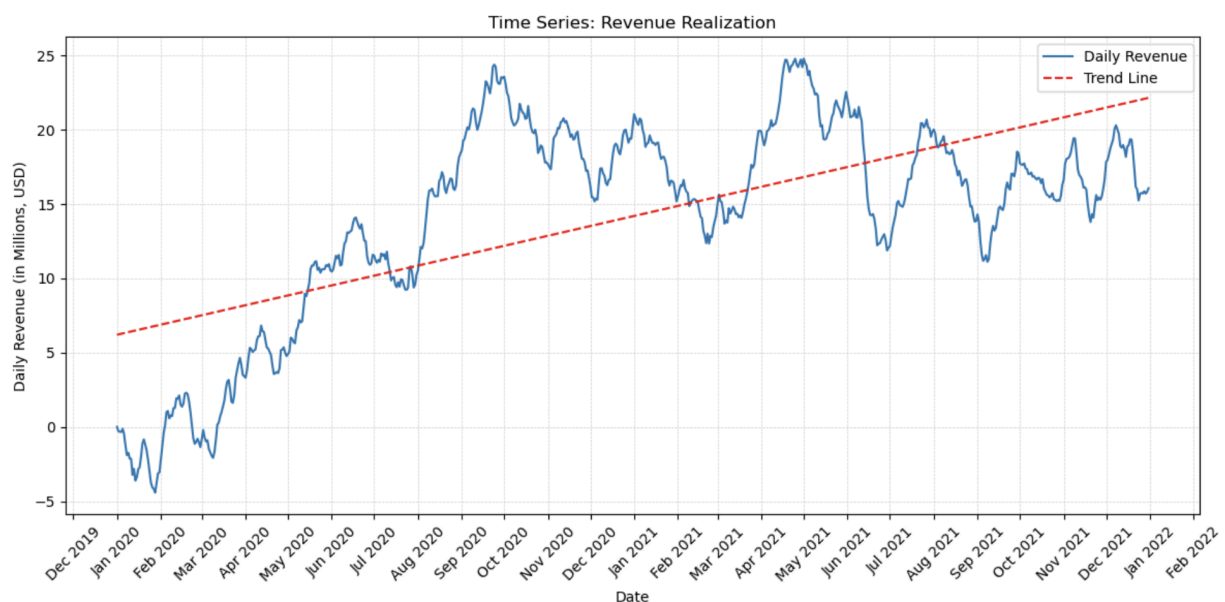
There are no missing dates - each consecutive row in the dataset is exactly 1 day apart. This means the dataset has complete daily coverage with **no gaps** or skipped observations.

Length of the Sequence

The time series spans **731 days**, which is roughly two years (there is an additional day due to 2020 being a leap year).

C3. Stationarity Evaluation

Looking solely at the time series line plot we generated in section C1 (shown below), we see an obvious upward trend. Given that non-stationary time series typically exhibit trends, seasonality, or changing variances (Arif, 2024), this clear trendline indicates that our data is **not stationary**.



The Augmented Dickey-Fuller (ADF) test can also be used to determine if a time series is stationary. The null hypothesis (H_0) of the ADF test assumes that a unit root is present in the time series data, meaning it is non-stationary. The alternative hypothesis (H_1) is that the data is stationary. A test statistic, which is a negative value, is calculated using the ADF test. The more negative the test statistic, the stronger the evidence against the null hypothesis and for stationarity. The test also provides a p-value, which is the probability of observing the test statistic if the null hypothesis is true. If the p-value is less than the significance level (0.05), the

null hypothesis is rejected, and the data is likely stationary. The ADF test also provides critical values at different significance levels. If the test statistic is less than or equal to the critical value, the null hypothesis is rejected, indicating the data is stationary (Vijay, 2025).

```
# Perform Augmented Dickey-Fuller (ADF) test
adf_result = adfuller(df)

# ADF test components
adf_stat = adf_result[0] # test stat
p_value = adf_result[1] # p-value
critical_values = adf_result[4] # crit values

# Display ADF test results
print("Augmented Dickey-Fuller Test Results:\n -----")
print(f"• ADF Statistic: {adf_stat:.4f}")
print(f"• p-value: {p_value:.4f}")
print(f"• Critical Values:")
for key, value in critical_values.items():
    print(f"    • {key} level: {value:.4f}")
print("-----")
```

Augmented Dickey-Fuller Test Results:

- ADF Statistic: -2.2183
- p-value: 0.1997
- Critical Values:
 - 1% level: -3.4394
 - 5% level: -2.8655
 - 10% level: -2.5689

Performing an ADF test using the `adfuller()` function from `Statsmodels` (the results of which are shown above), we generate a p-value of 0.1997. This value is greater than the significance threshold of 0.05, meaning we fail to reject the null hypothesis of the ADF test. This suggests that the revenue time series is non-stationary. Looking also at our ADF test statistic of -2.2183 against the critical values, we find that the test statistic is less than all critical values listed. This provides additional evidence to reject the null hypothesis, further implying that our series is **not stationary**.

C4. Data Preparation Steps

Step 1. Convert 'Day' Column to 'Date' (DateTime object)

The course provided time series dataset records daily revenue, in million dollars (which will be assumed to be USD), during the first two years of operation. Unfortunately, our data dictionary neglects to mention what years this data is taken from, however, it does include the presence of 731 daily observations, indicating the dataset includes data from a leap year (since there are 365 days in a year, a normal 2-year span would mean there would be only 730 observations). Assuming this would be relatively recent information and the last known leap year occurred in 2020, we'll provide a start date of 01/01/2020 for this information. This is done by creating a new `DateTime` object, representing our start date of '2020-01-01'. We'll use this object to convert our initial 'Day' column from random values (e.g., 0, 1, 2) to actual dates (e.g., 2020-01-01, 2020-01-02, 2020-01-03). We'll then rename our columns to reflect this change (replacing 'Day' with 'date') and update the 'Revenue' column label to reflect standard pythonic naming conventions (changing it to lowercase: 'revenue'). The new 'date' column will finally be set as the index of the `DataFrame` (`df`) so our dataset will be treated as chronological (time series) data.

```
# Create a new datetime object, representing a start date of 01/01/2020 (the most recent 2-year range that includes
start_date = pd.to_datetime('2020-01-01')

# Convert 'Day' column to actual dates|
df['Day'] = pd.to_timedelta(df['Day']-1, unit='D') + start_date

# Rename columns
df.columns = ['date', 'revenue']

# Set 'date' as the index, so our dataset will be treated as chronological data
df.set_index('date', inplace=True)

# Visually inspect final dataframe to verify appearance as expected
df
```

	revenue
date	
2020-01-01	0.000000
2020-01-02	-0.292356
2020-01-03	-0.327772
2020-01-04	-0.339987
2020-01-05	-0.124888
...	...
2021-12-27	15.722056
2021-12-28	15.865822
2021-12-29	15.708988
2021-12-30	15.822867
2021-12-31	16.069429

731 rows × 1 columns

Step 2. Check for Missing Values

To check for missing values in our dataset, we'll utilize the `isnull()` function, employing the `sum()` method to return the total number of missing values in each column.

```
# Check for missing values in our dataset
df.isnull().sum()

revenue    0
dtype: int64
```

As we can see from the result above, there appears to be no missing values in our dataset.

Step 3. Check for Outliers & Other Anomalies

Since we're working solely with continuous data, we'll use the `describe()` method to identify any potential outliers in our revenue variable, focusing on detecting data points that may indicate data entry errors or those that deviate significantly from the expected distribution.

```
# Summary statistics for revenue
df.describe()
```

	revenue
count	731.000000
mean	14.179608
std	6.959905
min	-4.423299
25%	11.121742
50%	15.951830
75%	19.293506
max	24.792249

Looking at summary statistics for daily revenue (shown above), it appears we have a mean of about \$14 million, a negative minimum daily revenue of approximately -\$4.42 million and a maximum daily revenue of \$24.79 million. The negative minimum value appears most concerning, so we'll use the `nlargest()` and `nsmallest()` methods to list the top 10 highest and lowest daily revenues by date (shown below). Doing this reveals that our negative minimum value isn't abnormal and that we have several other negative values across other days, indicating we have several days of revenue loss.

```
# Check for outliers in our revenue
revenue_largest = df['revenue'].nlargest(10)
revenue_smallest = df['revenue'].nsmallest(10)

print("LOWEST DAILY REVENUES BY DATE (Low to High):\n", revenue_smallest.to_string(index=True))
print("\nHIGHEST DAILY REVENUES BY DATE (High to Low):\n", revenue_largest.to_string(index=True))
```

LOWEST DAILY REVENUES BY DATE (Low to High):

date	
2020-01-28	-4.423299
2020-01-27	-4.172912
2020-01-26	-4.068318
2020-01-29	-3.773237
2020-01-25	-3.749676
2020-01-14	-3.609713
2020-01-15	-3.398875
2020-01-12	-3.228637
2020-01-30	-3.130366
2020-01-31	-3.066997

HIGHEST DAILY REVENUES BY DATE (High to Low):

date	
2021-05-01	24.792249
2021-04-25	24.786487
2021-04-29	24.737544
2021-04-18	24.719916
2021-04-19	24.702191
2021-04-24	24.592677
2021-04-28	24.504853
2021-05-02	24.502237
2021-04-26	24.401791
2020-09-24	24.390297

Step 4. Split the Dataset into Training & Test Sets

To keep the inherent sequential order of our time series data, we'll use an `iloc` split by index to split our dataset. First, we define the number of rows (days) to reserve for testing - which, corresponding to our research question, will be 120. Then, using `.iloc` for index-based selection, we select all rows except the last 120 to be used as the training set.

```
# Set fixed test size (120 days, in relation to research question)
test_size = 120

# Split into training and test sets
train = df.iloc[:-test_size]
test = df.iloc[-test_size:]
```

C5. Cleaned Dataset

The prepared datasets, 'd213_task1_train.csv' and 'd213_task1_test.csv', can be found within the main project directory and are also included as attachments to this report.

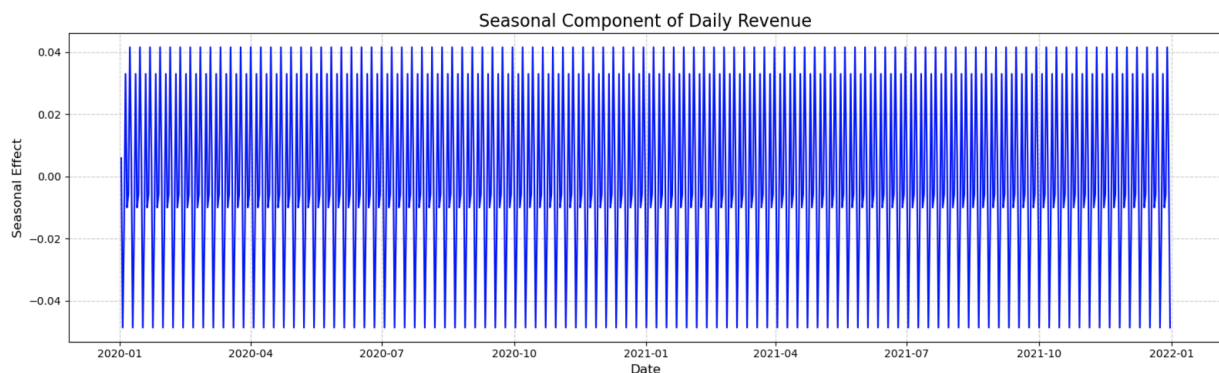
```
# Save training and test sets to CSV
train.to_csv('d213_task1_train.csv')
test.to_csv('d213_task1_test.csv')
```

Part IV: Model Identification & Analysis

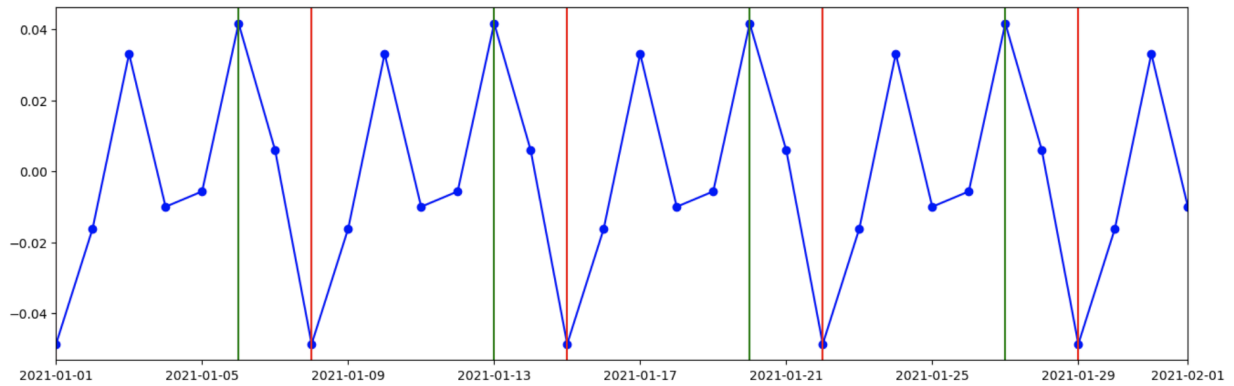
D1. Annotated Findings & Visualizations

1. Seasonality

Upon performing seasonal decomposition and plotting the seasonal component, we can see a clear, repetitive wave pattern indicating strong and consistent seasonality in the data. The pattern also appears to occur at regular intervals, suggesting cyclical trends in revenue.

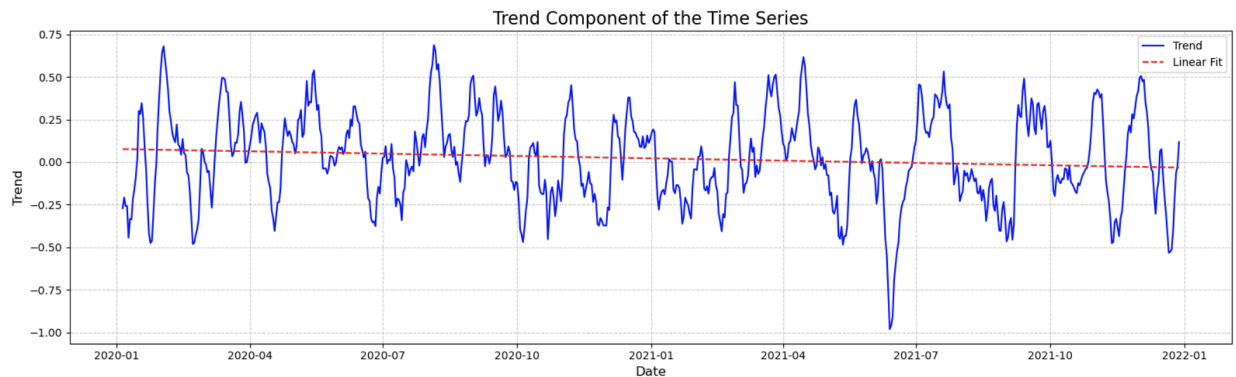


A closer look at the graph reveals a weekly seasonal pattern, with the highest revenue occurring on Wednesdays and the lowest on Fridays. This weekly seasonality suggests that the ARIMA model will need to be adjusted to account for this pattern, either by manually removing the seasonal component or by using a SARIMA or SARIMAX model (which are essentially more powerful, flexible versions of ARIMA).



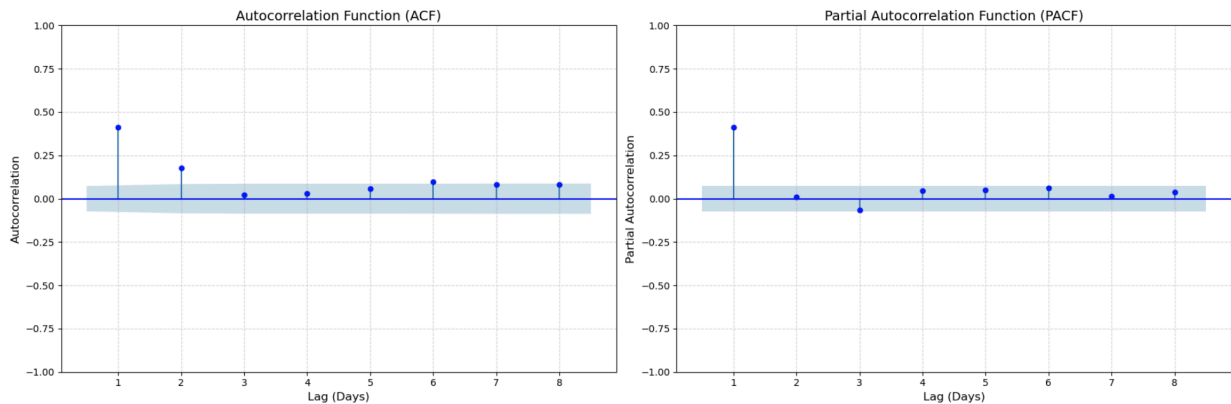
2. Trends

The slightly negative, downward slope of the trend component (the red dashed line in the image below) indicates a gradual decrease in daily revenue over the two-year period.



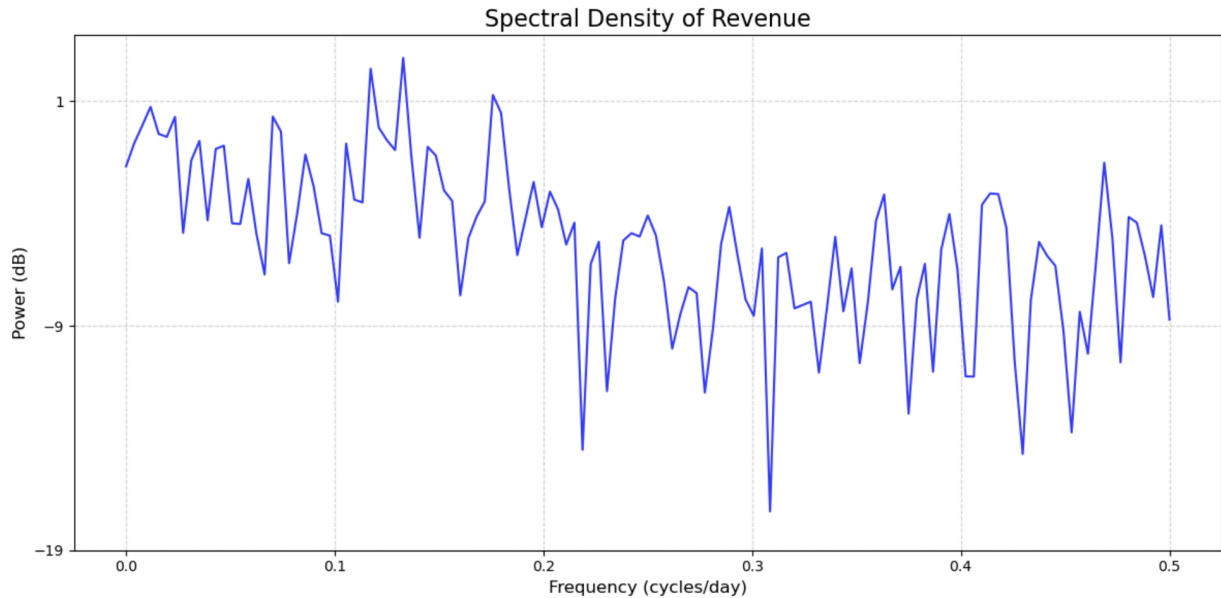
3. Autocorrelation Function (ACF)

The ACF plot (on the left) shows a strong spike at lag 1, well above the confidence interval (the light blue band). This observation suggests a substantial autocorrelation at a 1-day lag. The PACF plot (on the right) also displays a notable spike at lag 1, indicating a direct relationship between a day's revenue and the day prior.



4. Spectral Density

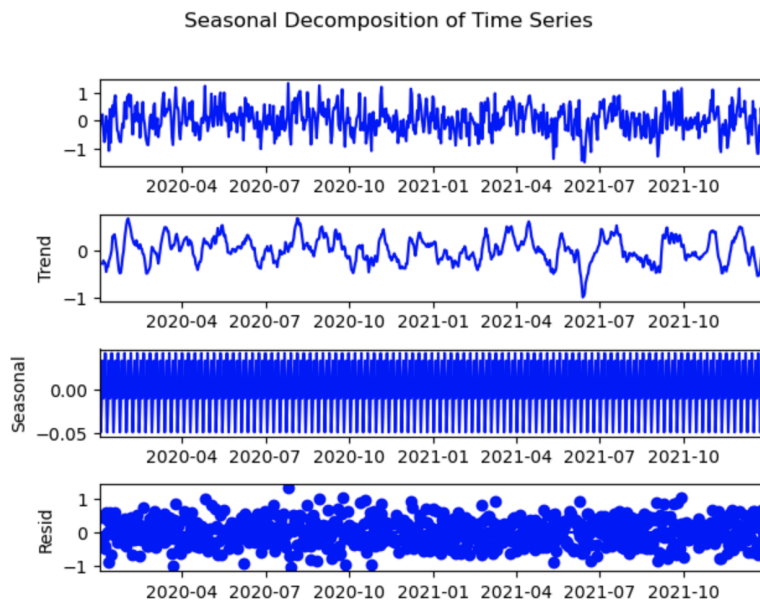
The spectral density plot reveals that power is concentrated in the lower frequencies, with the highest peak around 0.14 cycles/day. This finding points to a dominant frequency in that range. Interpreting this frequency in terms of days, we can calculate a corresponding period ($1 / 0.14 \approx 7.14$ days). This result aligns with the strong weekly seasonal pattern previously identified.



5. Decomposed Time Series

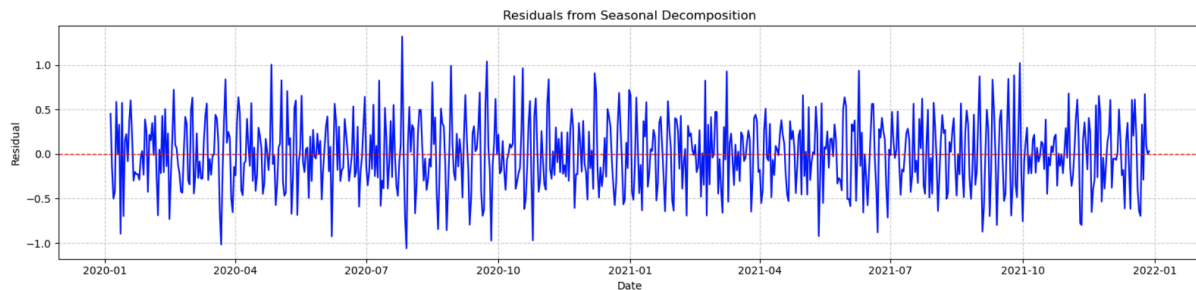
The top plot displays the original, observed data, and its repetitive pattern confirms the presence of seasonality. The second plot illustrates the long-term trend, with clear rising and falling fluctuations in revenue. A consistent, repeating pattern in the third plot confirms weekly seasonality, as we discussed earlier. In the bottom plot, residual values are centered around 0, without a visible trend, but with slight heteroscedasticity and greater variability in some periods,

suggesting the potential presence of outliers.



6. Confirmation of Lack of Trends in Residuals of Decomposed Series

The plot shows that residuals fluctuate randomly above and below the red zero line, indicating the seasonal decomposition has successfully removed both trend and seasonality from the original time series.



D2. ARIMA Model

```
# Initialize SARIMA model with weekly seasonality (period = 7)
model = SARIMAX(train['revenue'], order=(0, 1, 2), seasonal_order=(1, 1, 1, 7))
model_results = model.fit(dispatch=False)

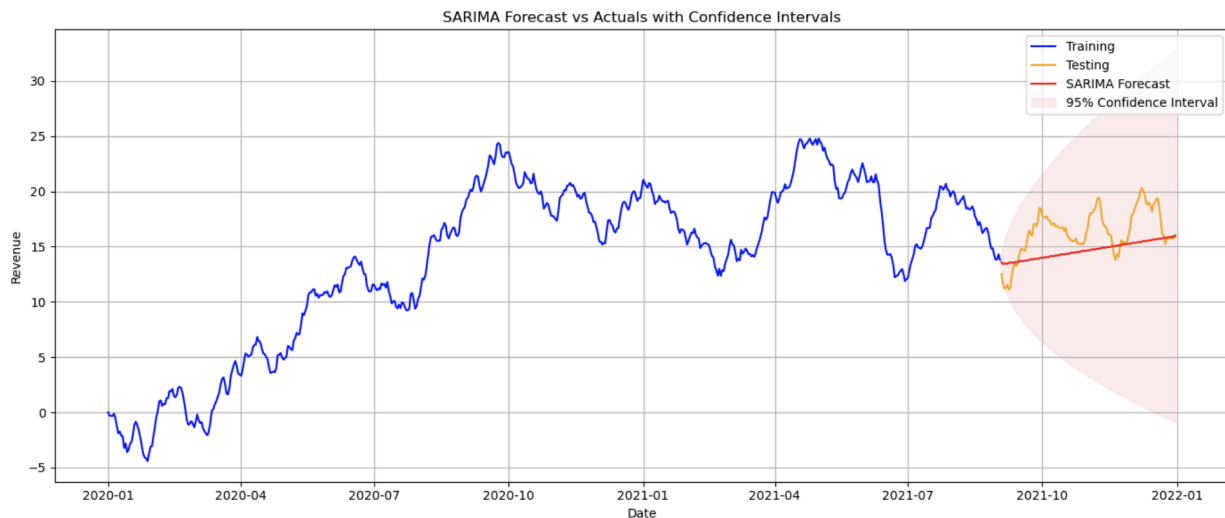
print(model_results.summary())
```

```
SARIMAX Results
=====
Dep. Variable:          revenue      No. Observations:          611
Model:                SARIMAX(0, 1, 2)x(1, 1, [1], 7)      Log Likelihood          -374.603
Date:                  Fri, 09 May 2025                    AIC              759.206
Time:                  14:57:50                             BIC              781.216
Sample:                01-01-2020                         HQIC              767.772
                    - 09-02-2021
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ma.L1          0.3874     0.040     9.635     0.000     0.309     0.466
ma.L2          0.2158     0.040     5.330     0.000     0.136     0.295
ar.S.L7        0.0236     0.043     0.555     0.579    -0.060     0.107
ma.S.L7       -0.9995     0.961    -1.040     0.298    -2.883     0.884
sigma2         0.1928     0.183     1.055     0.291    -0.165     0.551
=====
Ljung-Box (L1) (Q):          0.05   Jarque-Bera (JB):          2.56
Prob(Q):                   0.82   Prob(JB):          0.28
Heteroskedasticity (H):      1.01   Skew:          -0.05
Prob(H) (two-sided):         0.94   Kurtosis:         2.70
=====
```

```
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

As we saw in the previous section, the data exhibits strong weekly seasonality, which led to the selection of a Seasonal ARIMA (SARIMA) model with a period of 7 to accommodate. A non-seasonal ARIMA model was initially tested, but it consistently produced a flat forecast line, even after differencing to address non-stationarity, manually removing the seasonal component, and using grid search to optimize ARIMA parameters. After some experimentation, I ultimately decided a SARIMA model was the best type of ARIMA model for handling the seasonality present in our time series data. Please note the model summary shows SARIMAX, but it's actually fitting a SARIMA model since no exogenous terms are present.

D3. Forecast



The forecast graph displays the model's ability to predict hospital revenue over the next 120 days based on historical trends, capturing both overall direction and recurring seasonal patterns. The **blue line** represents the **training data** used to fit the model, while the **orange line** shows the actual revenue values from the **test set**. The **red line**, representing the SARIMA **forecast**, reflects the upward trend and general rhythm of the weekly seasonality observed in the training data, though with slightly smoother variation.

The pink shaded area illustrates the **95% confidence interval**, providing a range within which future revenue values are statistically expected to fall. Most actual revenue values in the test set lie within this confidence band, suggesting the forecast is both reliable and well-calibrated.

```
# Evaluate performance
mae = mean_absolute_error(y_test, y_forecast)
rmse = np.sqrt(mean_squared_error(y_test, y_forecast))

print(f"• Forecasting Model's MAE (Mean Absolute Error): {mae:.2f}")
print(f"• Forecasting Model's RMSE (Root Mean Squared Error): {rmse:.2f}")
```

- Forecasting Model's MAE (Mean Absolute Error): 2.05
- Forecasting Model's RMSE (Root Mean Squared Error): 2.46

The forecasting model demonstrates strong predictive accuracy, with a Mean Absolute Error (MAE) of 2.05 and a Root Mean Squared Error (RMSE) of 2.46. This suggests an average daily prediction error of approximately \$2.05 million. The low and similar values of both error metrics indicate consistent and accurate forecasts with few large deviations across the 120-day window, implying the model is reliable for short-term financial and operational planning.

D4. Output & Calculations of the Analysis

Note: The links in this section will take you to the corresponding sections within this report.

[C3. Stationarity Evaluation](#)

To perform our ADF test, the following formula is implemented using the `adfuller()` function from Statsmodels (Santra, 2023):

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \dots + \delta_{p-1} \Delta y_{t-p+1} + \varepsilon_t$$

Where:

- y_t is the value of the times are the time t
- Δy_t is the first difference ($y_t - y_{t-1}$)
- α is the constant (intercept), that captures drift
- βt is the time trend
- γy_{t-1} is the key term (tests for the unit root)
- $\delta_1 \Delta y_{t-1} + \dots + \delta_{p-1} \Delta y_{t-p+1}$ are the lagged differences (the “augmentation” to handle correlation)
- ε_t is the white noise error term

Augmented Dickey-Fuller (ADF) test:

```
adf_result = adfuller(df)
```

```
# ADF test components
```

```
adf_stat = adf_result[0] # test stat
```

```
p_value = adf_result[1] # p-value
```

```
critical_values = adf_result[4] # crit values
```

```
# Display ADF test results
```

```
print("Augmented Dickey-Fuller Test Results:\n -----")
```

```
print(f"• ADF Statistic: {adf_stat:.4f}")
```

```
print(f"• p-value: {p_value:.4f}")
```

```
print("• Critical Values:")
```

```
for key, value in critical_values.items():
```

```
    print(f" • {key} level: {value:.4f}")
```

```
print("-----")
```

Output of the ADF test used to evaluate the stationarity of the time series:

```
Augmented Dickey-Fuller Test Results:
```

```
-----
• ADF Statistic: -2.2183
• p-value: 0.1997
• Critical Values:
  • 1% level: -3.4394
  • 5% level: -2.8655
  • 10% level: -2.5689
-----
```

D1. Annotated Findings & Visualizations

1. Seasonality Plots (showing presence or lack of a seasonal component):

```
# Perform seasonal decomposition
```

```
decomp = seasonal_decompose(df_transformed)
```

```
# Plot the seasonal component
```

```
plt.figure(figsize = [16,5])
```

```
plt.plot(decomp.seasonal, color='blue')
```

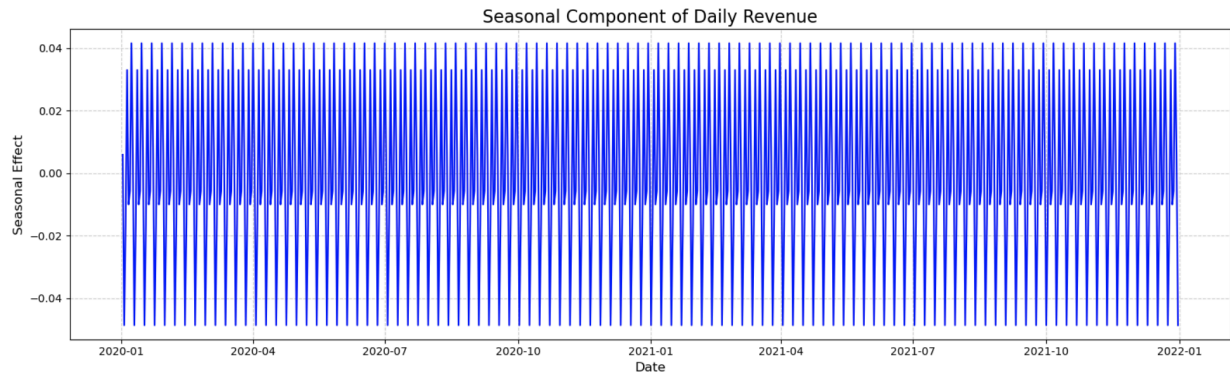
```
plt.title("Seasonal Component of Daily Revenue", fontsize=16)
```

```
plt.xlabel("Date", fontsize=12)
```

```
plt.ylabel("Seasonal Effect", fontsize=12)
```

```
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output of the Seasonal Component of Daily Revenue:

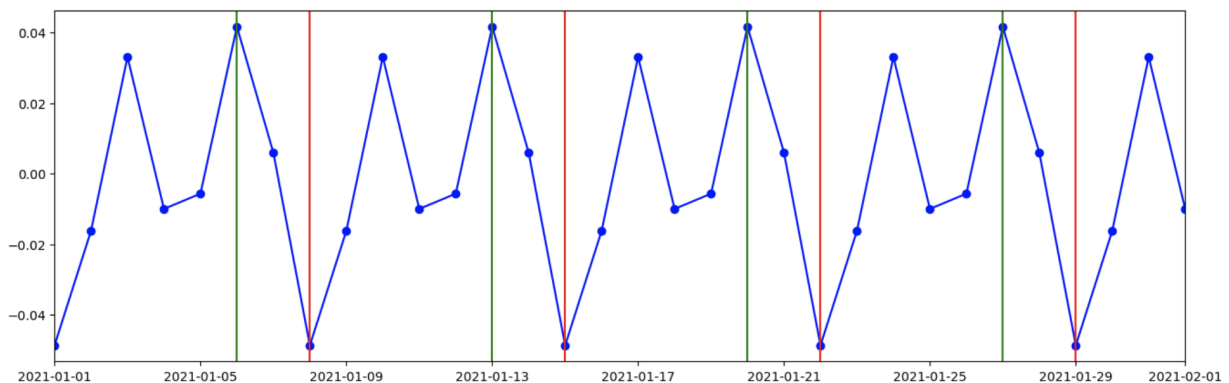


```
# Plot the seasonal component
plt.figure(figsize = [16,5])
plt.plot(decomp.seasonal, marker='o', color='blue')
plt.xlim(pd.to_datetime('2021-01-01'), pd.to_datetime('2021-02-01'))
```

```
# Draw red lines on Fridays (lowest peaks)
plt.axvline(x=pd.to_datetime('2021-01-01'), color='red')
plt.axvline(x=pd.to_datetime('2021-01-08'), color='red')
plt.axvline(x=pd.to_datetime('2021-01-15'), color='red')
plt.axvline(x=pd.to_datetime('2021-01-22'), color='red')
plt.axvline(x=pd.to_datetime('2021-01-29'), color='red')
```

```
# Draw green lines on Wednesdays (highest peaks)
plt.axvline(x=pd.to_datetime('2021-01-06'), color='green')
plt.axvline(x=pd.to_datetime('2021-01-13'), color='green')
plt.axvline(x=pd.to_datetime('2021-01-20'), color='green')
plt.axvline(x=pd.to_datetime('2021-01-27'), color='green')
```

Output of the Weekly Seasonal Component:



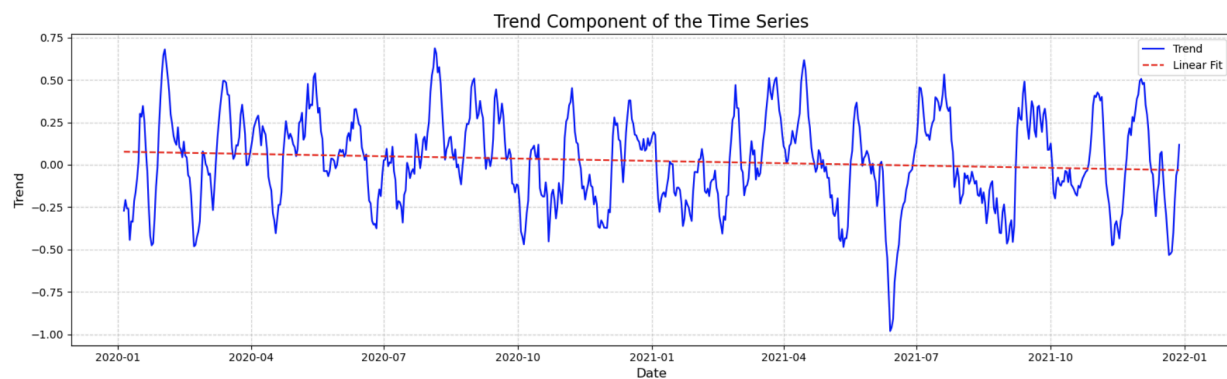
2. Trends Plot:

```
trend_clean = decomp.trend.dropna()
x_vals = mdates.date2num(trend_clean.index)
y_vals = trend_clean.values

# Fit linear regression line
z = np.polyfit(x_vals, y_vals, 1)
p = np.poly1d(z)

# Plot trend with regression line
plt.figure(figsize=(16,5))
plt.plot(trend_clean.index, trend_clean, label='Trend', color='blue')
plt.plot(trend_clean.index, p(x_vals), 'r--', label='Linear Fit')
plt.title("Trend Component of the Time Series", fontsize=16)
plt.xlabel("Date", fontsize=12)
plt.ylabel("Trend", fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output of the trend component of the time series:

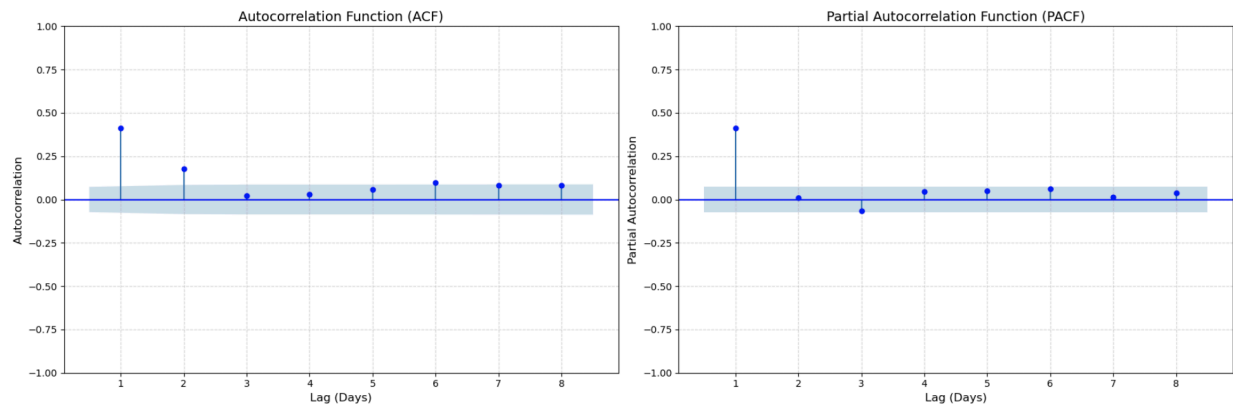


3. Autocorrelation Function (ACF) Plot:

```
# ACF plot
fig, ax = plt.subplots(1, 2, figsize=(18, 6))
plot_acf(df_transformed, lags=8, ax=ax[0], color='blue', zero=False)
ax[0].set_title("Autocorrelation Function (ACF)", fontsize=14)
ax[0].set_xlabel("Lag (Days)", fontsize=12)
ax[0].set_ylabel("Autocorrelation", fontsize=12)
ax[0].grid(True, linestyle='--', alpha=0.5)

# PACF plot
plot_pacf(df_transformed, lags=8, ax=ax[1], method='ywm', color='blue', zero=False)
ax[1].set_title("Partial Autocorrelation Function (PACF)", fontsize=14)
ax[1].set_xlabel("Lag (Days)", fontsize=12)
ax[1].set_ylabel("Partial Autocorrelation", fontsize=12)
ax[1].grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

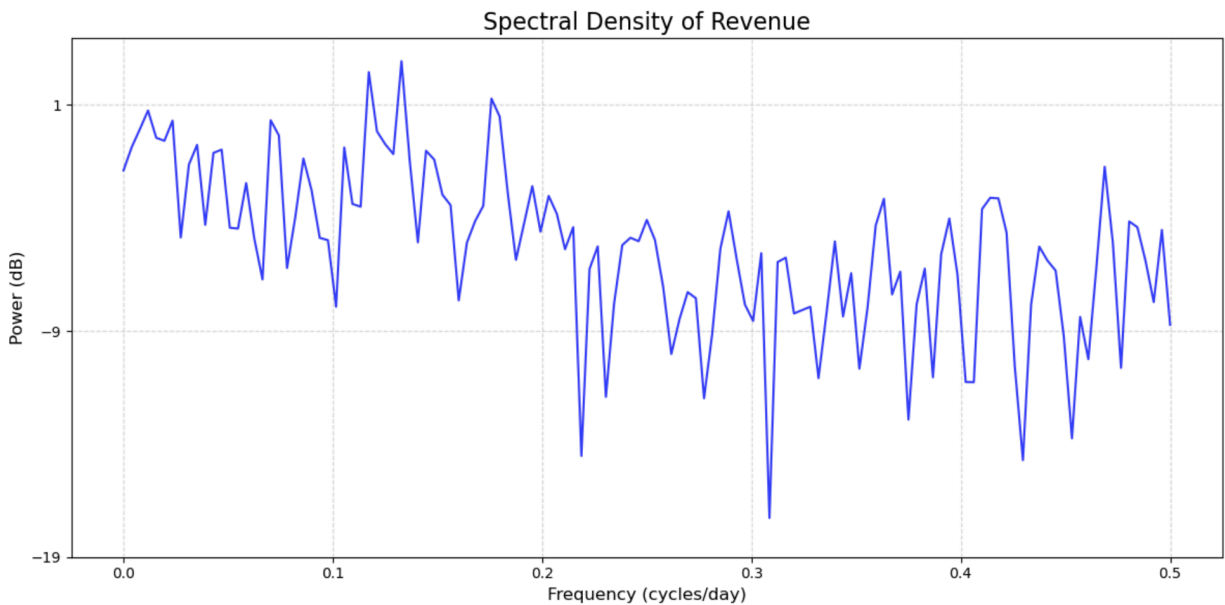
Output of the autocorrelation of revenue:



4. Spectral Density Plot:

```
plt.figure(figsize=(12, 6))
plt.psd(df_transformed['revenue'], NFFT=256, Fs=1, color='blue', linewidth=1.5, alpha=0.8)
plt.title('Spectral Density of Revenue', fontsize=16)
plt.xlabel('Frequency (cycles/day)', fontsize=12)
plt.ylabel('Power (dB)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

Output of the spectral density of the revenue time series:

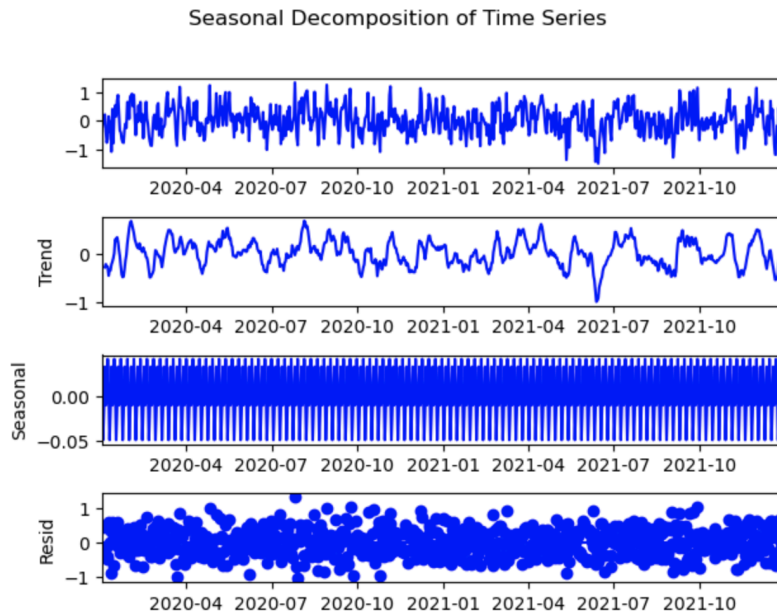


5. Decomposed Time Series:

```
fig = decomp.plot()
for ax in fig.axes:
    for line in ax.get_lines():
        line.set_color("blue")
```

```
plt.suptitle("Seasonal Decomposition of Time Series", y=1.02)
plt.tight_layout()
plt.show()
```

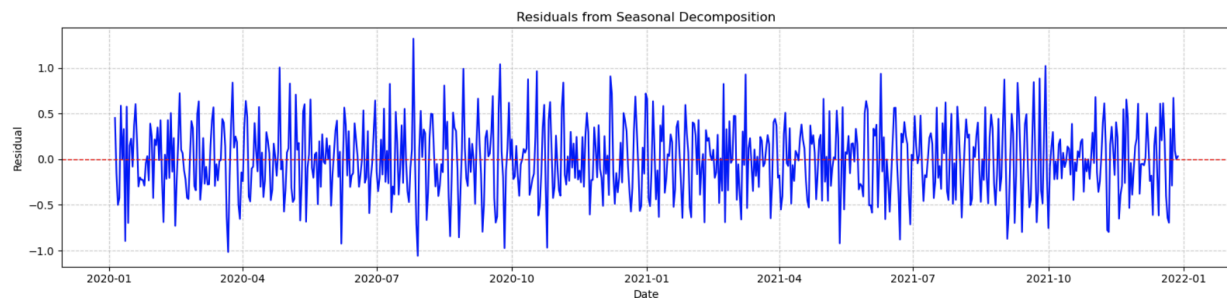
Output of the decomposition of the time series:



6. Residuals of Decomposition:

```
plt.figure(figsize=(16, 4))
plt.plot(decomp.resid, color='blue', label='Residuals')
plt.axhline(0, color='red', linestyle='--', linewidth=1)
plt.title("Residuals from Seasonal Decomposition")
plt.xlabel("Date")
plt.ylabel("Residual")
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output of the residuals of decomposition:



D2. ARIMA Model

```
# Initialize SARIMA model with weekly seasonality (period = 7)
model = SARIMAX(train['revenue'], order=(0, 1, 2), seasonal_order=(1, 1, 1, 7))
model_results = model.fit(dispatch=False)
```

```
print(model_results.summary())
```

Output of the final model summary:

SARIMAX Results						
Dep. Variable:	revenue		No. Observations:	611		
Model:	SARIMAX(0, 1, 2)x(1, 1, [1], 7)		Log Likelihood	-374.603		
Date:	Fri, 09 May 2025		AIC	759.206		
Time:	15:28:24		BIC	781.216		
Sample:	01-01-2020		HQIC	767.772		
	- 09-02-2021					
Covariance Type:	opg					
	coef	std err	z	P> z	[0.025	0.975]
ma.L1	0.3874	0.040	9.635	0.000	0.309	0.466
ma.L2	0.2158	0.040	5.330	0.000	0.136	0.295
ar.S.L7	0.0236	0.043	0.555	0.579	-0.060	0.107
ma.S.L7	-0.9995	0.961	-1.040	0.298	-2.883	0.884
sigma2	0.1928	0.183	1.055	0.291	-0.165	0.551
Ljung-Box (L1) (Q):	0.05		Jarque-Bera (JB):	2.56		
Prob(Q):	0.82		Prob(JB):	0.28		
Heteroskedasticity (H):	1.01		Skew:	-0.05		
Prob(H) (two-sided):	0.94		Kurtosis:	2.70		

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

D3. Forecast:

```
forecast = model_results.get_forecast(steps=len(test))
```

```
model_pred_mean = forecast.predicted_mean
```

```
model_conf_int = forecast.conf_int()
```

```
# Convert to arrays for plotting
```

```
x = np.array(test.index)
```

```
y_forecast = model_pred_mean.to_numpy()
```

```
y_lower = model_conf_int.iloc[:, 0].to_numpy()
```

```
y_upper = model_conf_int.iloc[:, 1].to_numpy()
```

```
y_test = test['revenue'].to_numpy()
```

```
y_train = train['revenue'].to_numpy()
```

```
x_train = np.array(train.index)
```

```
# Plot the forecast
```

```
plt.figure(figsize=(14, 6))
```

```
plt.plot(x_train, y_train, label='Training', color='blue')
```

```
plt.plot(x, y_test, label='Testing', color='orange')
```

```
plt.plot(x, y_forecast, label='SARIMA Forecast', color='red')
```

```
plt.fill_between(x, y_lower, y_upper, color='pink', alpha=0.3, label='95% Confidence Interval')
```

```
plt.xlabel('Date')
```

```
plt.ylabel('Revenue')
```

```
plt.title('SARIMA Forecast vs Actuals with Confidence Intervals')
```

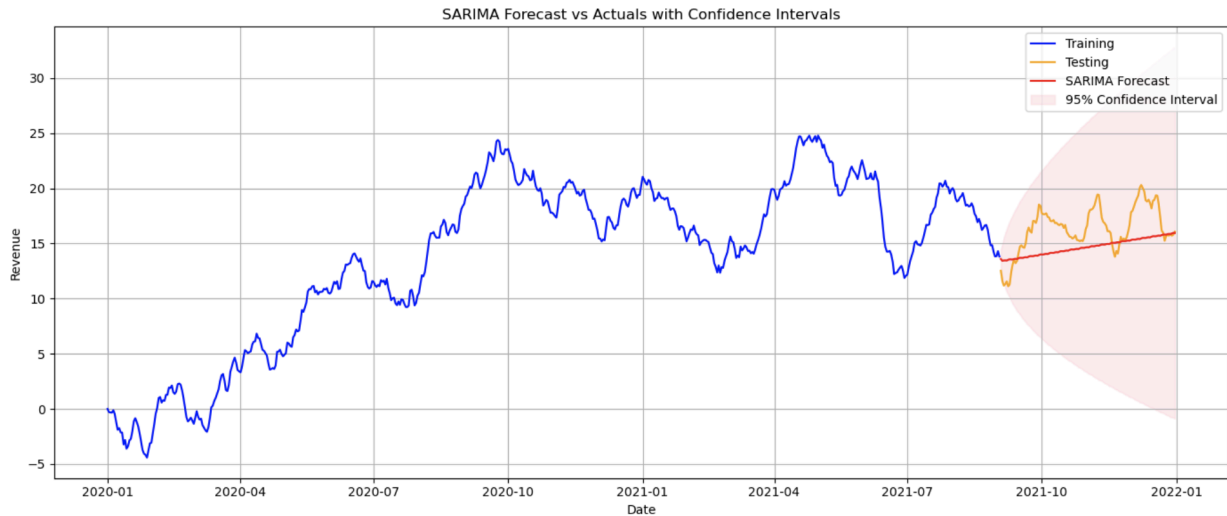
```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

Output of the forecast against actual values:



To evaluate our forecast further, the Mean Absolute Error (MAE) was calculated, implementing the following formula using the `mean_absolute_error()` function from Scikit-Learn (Deepchecks, 2024):

$$\text{MAE} = (1/n) \times \sum |y_i - \hat{y}_i| \text{ from } i = 1 \text{ to } n$$

Where:

- n is the number of observations in the dataset
- y_i is the true value
- $\hat{y}_i$ is the predicted value

We calculated an additional metric, the Root Mean Squared Error (RMSE), using the `mean_squared_error()` function from Scikit-Learn to determine the MSE, and taking the square root of the MSE value using the `sqrt()` function from NumPy, following the general formula for RMSE (KoshurAI, 2024):

$$\text{RMSE} = \text{sqrt} \left((1/n) \times \sum (y_i - \hat{y}_i)^2 \right) \text{ for } i = 1 \text{ to } n$$

Where:

- n is the total number of observations (data points) in the dataset
- y_i denotes the predicted (forecasted) value at time i
- $\hat{y}_i$ represents the actual value at time i
- $(y_i - \hat{y}_i)^2$ is the squared error

Forecast Evaluation:

```
mae = mean_absolute_error(forecast_df['actual'], forecast_df['forecast'])
rmse = np.sqrt(mean_squared_error(forecast_df['actual'], forecast_df['forecast']))
```

```
print(f"• Forecasting Model's MAE (Mean Absolute Error): {mae:.2f}")
print(f"• Forecasting Model's RMSE (Root Mean Squared Error): {rmse:.2f}")
```

Output of the forecast evaluation using MSE and RMSE:

- Forecasting Model's MAE (Mean Absolute Error): 2.05
- Forecasting Model's RMSE (Root Mean Squared Error): 2.46

D5. Code

Please refer to the attached file '**d213_task1.ipynb**' for all of the code used to support the implementation of the time series model.

Part V: Data Summary & Implications

E1. Data Analysis Results

Selection of an ARIMA Model

The SARIMA model was selected because the dataset of daily hospital revenue over two years displays both non-stationarity (confirmed by the Augmented Dickey-Fuller test performed in section C3) and weekly seasonality (shown in the decomposition plots in section D1). This justified the use of a SARIMA model with the following parameters: order=(0, 1, 2), seasonal_order=(1, 1, 1, 7). The d=1 and D=1 account for trend and seasonal differencing, while the seasonal lag of 7 captures weekly fluctuations in hospital revenue. Using the SARIMAX class (even without exogenous regressors) allowed us to flexibly fit this seasonal ARIMA model.

Prediction Interval of the Forecast

The prediction interval of the forecast is 120 steps into the future, corresponding to the number of days in the test set (relating back to our initial research question). Rather than using a standard percentage-based split, we instead applied a fixed split to the original two-year dataset (731 daily entries), designating the final 120 days as the test set and the preceding 611 days as the training set. This means our SARIMA model was trained on revenue data from 01-01-2020 through 08-06-2021, and was used to forecast daily revenue for the 120-day period spanning 08-07-2021 to 12-04-2021. The resulting forecast includes a 95% prediction interval, which provides a range of values within which the true revenue is expected to fall with 95% confidence.

Justification of the Forecast Length

The forecast covers a test period of 120 days, representing the final portion of a 731-day dataset. This fixed-length horizon is justifiable for several reasons. First, it aligns with the research objective of evaluating short-term revenue trends over approximately four months - a practical and relevant window for operational planning in a healthcare setting. Second, a

120-day forecast is long enough to capture multiple cycles of weekly seasonality, allowing the model to demonstrate its ability to generalize recurring revenue patterns. Lastly, reserving the remaining 611 days for training ensures the model has ample historical data to learn from, while keeping the test set realistic in scope for decision-making use cases such as budget forecasting, readmission mitigation, and resource allocation.

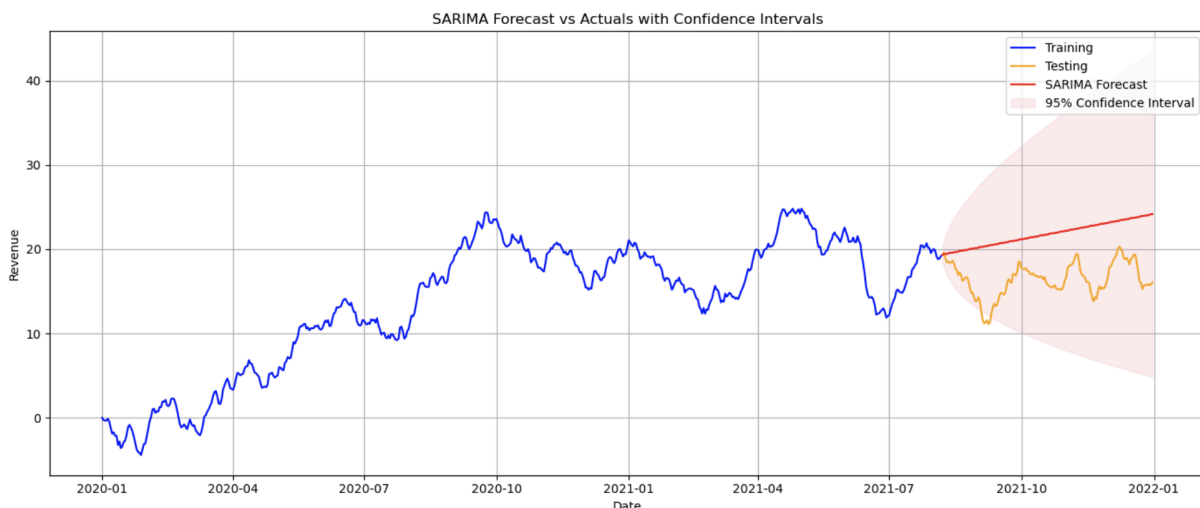
Model Evaluation Procedure & Error Metric

The model was evaluated using two common error metrics, Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). MAE is straightforward to interpret - it measures the average magnitude of forecasting errors in the same units as the data (Pandelu, 2024). In our case, the MAE was 2.05, meaning the model's predicted revenue was off by an average of approximately \$2.05 million per day. This makes MAE particularly useful for understanding the typical error experienced during the 120-day forecast window.

RMSE, which was calculated at 2.46, penalizes larger errors more heavily by squaring them before averaging. The fact that our RMSE value is only slightly greater than MAE suggests that while some larger deviations occurred, they weren't extreme or frequent enough to significantly distort the overall accuracy. This relatively small gap indicates consistent performance across the test period.

Together, MAE and RMSE offer a balanced and complementary view of the model's forecasting accuracy. They help quantify how closely the forecast aligns with actual hospital revenue trends, directly supporting the research question of whether the model can be trusted to predict revenue over the next 120 days with confidence.

E2. Visualization of the Forecast of the Final Model



The **blue line** represents the **training data** used to fit the model, while the **orange line** shows the **actual revenue values from the test set**. The **red line**, representing the **SARIMA forecast**, closely tracks the seasonality observed in the training data, demonstrating that the model effectively identified recurring weekly patterns. This is a significant improvement over a standard ARIMA model, which produced a flat, less informative forecast.

The **pink shaded area** illustrates the **95% confidence interval**, providing a range within which the true values are expected to fall. Most actual test data points lie within this band, suggesting the forecast is both reliable and well-calibrated.

E3. Recommended Course of Action

Circling back to our research question of “What is the expected pattern of hospital revenue over the next 120 days, and how confident can we be in those predictions?” Using a SARIMA model trained on historical revenue data, we forecasted revenue for the 120-day period spanning from August 7, 2021 to December 4, 2021. .

The resulting forecast revealed a clear weekly seasonal pattern, indicating that hospital revenue follows consistent cyclical trends (likely tied to operational or patient visit patterns). The model's predictions closely followed the actual revenue values during the test period, with most observed data points falling within the 95% confidence interval. This suggests a high level of confidence in the model's predictive ability.

Performance metrics further support this conclusion, indicating a reasonable level of accuracy given the revenue scale. Overall, the model effectively answered the research question by providing a trustworthy short-term revenue forecast with quantifiable confidence. However, there's room for improvement in capturing the full complexity and volatility of the actual data.

The SARIMA model didn't include external drivers of revenue. To better capture volatility and improve responsiveness, we may want to move to a SARIMAX model, which can allow us to include exogenous variables, considering additional features such as day of the week, holidays, patient admission behavior (such as the influx of patients around cold/flu season), insurance billing cycle indicators, and other external factors that might explain variations that the time series alone could not.

Part VI: Reporting

F. Jupyter Notebook Report

Please refer to the attached file '**d213_task1.html**' for the HTML document of the executed Jupyter Notebook Presentation.

G. Data & Code Sources

Western Governors University. (2023). *Medical Data and Dictionary Files* [Dataset]. Data Files and Associated Dictionary Files.

Retrieved March 27, 2025, from

<https://access.wgu.edu/ASP3/aap/content/g9rke9s0rlc9ejd92md0.html>

H. References

- Arif, A. (2024, June 17). *Stationary Time-Series Analysis*. Timescale.
Retrieved April 1, 2025 from timescale.com/learn/stationary-time-series-analysis
- Deepchecks. (2024, May 27). *Mean Absolute Error*. Deepchecks Glossary.
Retrieved April 4, 2025 from deepchecks.com/glossary/mean-absolute-error
- EdX. (2022, May 26). *What Is ARIMA Modeling?* EdX.
Retrieved April 3, 2025 from mastersindatascience.org/learning/statistics-data-science/what-is-arima-modeling
- KosherAI. (2024, May 15). *Demystifying RMSE: Your Guide to Understanding Root Mean Squared Error*. Medium.
Retrieved April 4, 2025 from kosherai.medium.com/demystifying-rmse-your-guide-to-understanding-rmse
- Kuo, A. (2024, June 12). *What is Time Series Analysis?* Julius.
Retrieved April 15, 2025 from julius.ai/articles/what-is-time-series-analysis
- Noble, J. (2024, May 24). *Introducing ARIMA Models*. IBM.
Retrieved April 3, 2025 from ibm.com/think/topics/arima-model
- Pandelu, A. (2024, November 6). *Day 10: Evaluation Metrics for Regression - MSE, MAE, RMSE, R^2 Score*. Medium.
Retrieved March 27, 2025 from medium.com/@bhatadithya54764118/day-10-evaluation-metrics-for-regression
- Pandian. (2025, March 6). *Time Series Analysis: Definition, Components, Methods, and Applications*. Analytics Vidhya.
Retrieved March 27, 2025 from analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-to-time-series-analysis
- Santra, R. (2023, May 11). *Tests for Stationarity in Time Series - Dickey Fuller Test & Augmented Dickey Fuller (ADF) Test*. Medium.
Retrieved April 4, 2025 from medium.com/@ritusantra/tests-for-stationarity-in-time-series-dickey-fuller-test
- Statistics Solutions. (2021, August 18). *Time Series Analysis*. Statistics Solutions.
Retrieved April 4, 2025 from statisticssolutions.com/free-resources/directory-of-statistical-analyses
- Vijay. (2025, March 10). *Understanding Stationary Time Series Analysis*. Analytics Vidhya.
Retrieved March 27, 2025 from analyticsvidhya.com/blog/2021/06/statistical-tests-to-check-stationarity-in-time-series
- Wainaina, P. (2023, October 24). *The Complete Guide to Time Series Forecasting Models*. Medium.
Retrieved April 15, 2025 from medium.com/@wainaina.pierre/the-complete-guide-to-time-series-forecasting-models